AArch64 Memory & Cache System — Technical Reference Workbook (based on arm64 Cortex A72) 1
Chapter 1 — Architectural Foundations: Visibility Domains, Memory Types, and CacheMaintenance Rationale 1
1. Scope 1
2. Visibility Domains 1
2.1 CoreLocal Domain (L1D / L1I) 1
2.2 ClusterShared Domain (L2 = PoU) 1
2.3 SystemWide Domain (PoC) 2
3. Architectural Rationale for Cache Maintenance 2
4. Memory Attribute System (MAIR, TCR, TTBR) 2
4.1 Normal, Cacheable 2
4.2 Normal, NonCacheable 3
4.3 DevicenGnRnE 3
4.4 WriteThrough 3
5. Architectural vs Microarchitectural Behavior 3
5.1 Architectural 3
5.2 Microarchitectural 3
6. Architectural Mental Model 4
7. Summary 4
Exercise 1.1 — Visibility Domain Identification 4
Exercise 1.2 — Attribute Impact 5
Exercise 1.3 — PoU vs PoC 5

AArch64 Memory & Cache System — Technical Reference Workbook (based on arm64 Cortex A72)

Chapter 1 — Architectural Foundations: Visibility Domains, Memory Types, and Cache-Maintenance Rationale

1. Scope

This chapter defines the architectural concepts required to understand AArch64 cache maintenance operations. It describes visibility domains, memory types, and the architectural motivation for DC/IC instructions and barrier semantics. All content is aligned with ARMv8-A architecture behavior and Cortex-A72/A73 microarchitectural characteristics.

2. Visibility Domains

AArch64 defines multiple memory-visibility domains. These domains determine when updates performed by one agent (CPU, DMA, other cores) become observable to other agents.

2.1 Core-Local Domain (L1D / L1I)

- Private to each core
- Non-coherent
- Contains data and instructions not visible to other cores or external agents
- May contain stale or dirty lines
- Not a point of architectural visibility

2.2 Cluster-Shared Domain (L2 = PoU)

- Shared by all cores in the cluster

- Coherent within the cluster
- Instruction refills originate from this domain
- First domain where instruction and data streams unify
- Architecturally defined as **Point of Unification (PoU)**

2.3 System-Wide Domain (PoC)

- Visible to DMA engines, other clusters, and the memory controller
- Architecturally defined as **Point of Coherence (PoC)**
- Represents the globally coherent memory domain

3. Architectural Rationale for Cache Maintenance

Cache maintenance instructions exist to ensure correct propagation of data and instructions across visibility domains. They are required because:

- L1D and L1I are private and non-coherent
- L2 is coherent only within the cluster
- DMA engines observe memory only at PoC
- Instruction fetches observe memory only at PoU
- Dirty lines may reside in L1D or L2
- Stale instructions may reside in L1I or the front-end pipeline

Cache maintenance operations ensure that:

- Data becomes visible at PoU or PoC
- Stale instructions are invalidated
- Memory ordering is enforced
- Multi-core and DMA interactions behave deterministically

4. Memory Attribute System (MAIR, TCR, TTBR)

Memory attributes determine how memory regions interact with the cache hierarchy and whether cache maintenance operations have architectural effect.

4.1 Normal, Cacheable

- Full caching permitted

- Dirty lines may exist
- DC/IC operations meaningful
- PoU and PoC semantics apply

4.2 Normal, Non-Cacheable

- No caching
- No dirty lines
- DC/IC operations architecturally ignored
- PoU/PoC semantics do not apply

4.3 Device-nGnRnE

- No caching
- Strong ordering rules
- DC/IC operations ignored
- Only barrier instructions enforce ordering

4.4 Write-Through

- No dirty lines
- Clean operations have no effect
- Invalidate operations may still be meaningful

Correct configuration of MAIR, TCR, and TTBR registers is a prerequisite for correct cache behavior.

5. Architectural vs Microarchitectural Behavior

5.1 Architectural

- Defined by ARMv8-A architecture
- Includes DC/IC semantics, PoU/PoC definitions, and barrier behavior
- Software-visible and deterministic

5.2 Microarchitectural

- Implementation-specific

- Includes FEQ (Fill/Evict Queue), snoop timing, internal pipelines, and register slices
- Hidden behind architectural barriers
- Not visible to software

Barrier instructions (DSB/ISB) ensure that microarchitectural pipelines do not affect architectural correctness.

6. Architectural Mental Model

The following model is used throughout the reference:

DC instructions propagate data between visibility domains. IC instructions invalidate stale instruction lines. DSB/ISB enforce architectural completion and visibility.

This model is consistent across all Cortex-A72/A73 cache-maintenance sequences.

7. Summary

- AArch64 defines multiple visibility domains: L1D/L1I (private), L2 (PoU), and PoC (system-wide).
- Cache maintenance instructions exist to propagate data and instructions across these domains.
- Memory attributes determine whether cache maintenance operations have architectural effect.
- Architectural barriers ensure completion and ordering of cache maintenance operations.
- Microarchitectural details are hidden behind architectural guarantees.

Exercise 1.1 — Visibility Domain Identification

Identify the visibility domain for each scenario:

1. CPU writes to L1D
2. CPU1 updates instructions for CPU2
3. DMA reads a buffer prepared by CPU
4. CPU reads a buffer written by DMA

Exercise 1.2 — Attribute Impact

Explain why DC/IC operations have no effect on Device-nGnRnE memory.

Exercise 1.3 — PoU vs PoC

Describe the difference between PoU and PoC in terms of instruction and data visibility.